

CS101C: Introduction to Programming (Using C)

Instructors: Prof. Aikta Arya and Prof. Konjengbam Anand

CSE Department

Indian Institute of Technology, Dharwad

Week 9

Today's Class (5/10/2026)

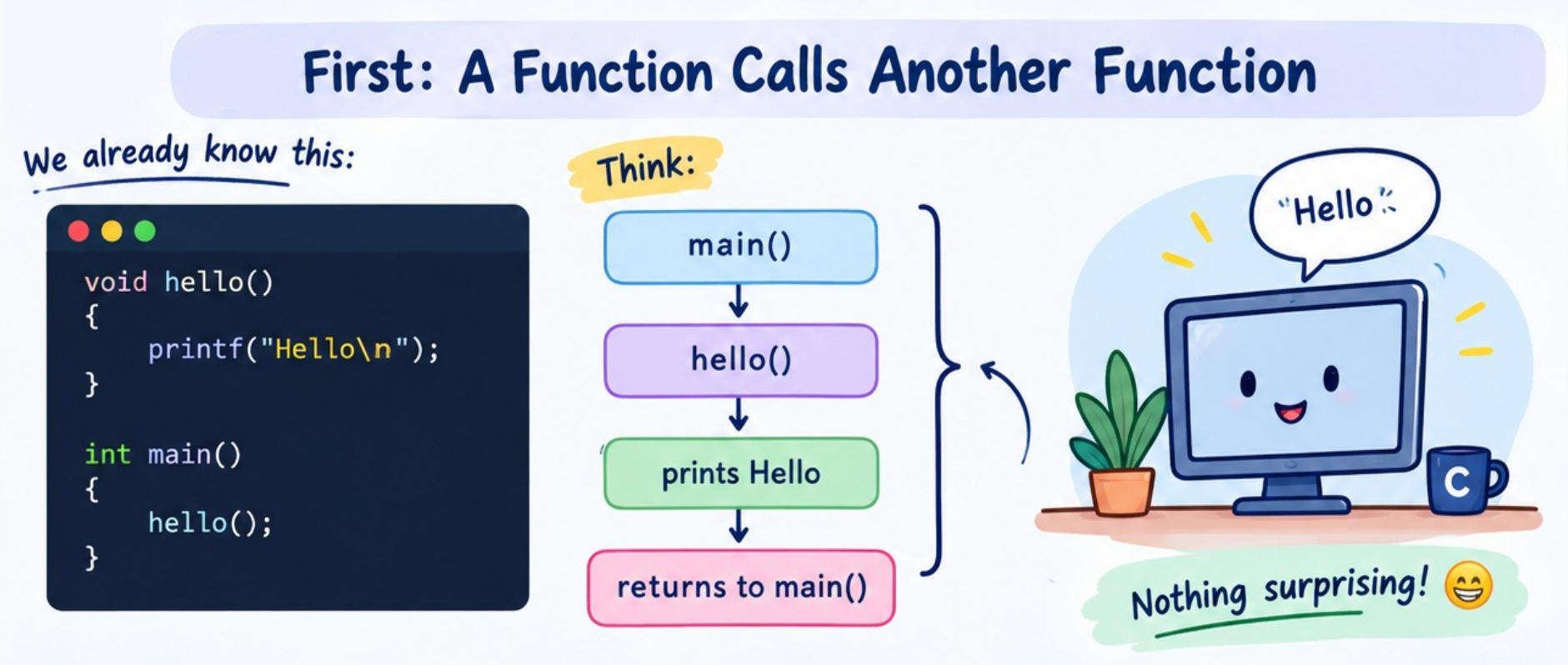
Recursion: The Function That Calls Itself

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad



First: A Function Calls Another Function

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad



First: A Function Calls Another Function

॥ सा विद्या या विमुक्तये ॥
य प्रौद्योगिकी संस्थान धारवाड
itute of Technology Dharwad

What if:

```
void hello()
{
    printf("Hello\n");
    hello();
}
```

and:

```
int main()
{
    hello();
}
```



The call sequence:

hello()
↓
hello()
↓
hello()
↓
hello()
↓
hello()
↓
• • •

PREDICT!

Will the program:

- A Print Hello once
- B Print Hello 10 times
- C Keep calling forever
- D Give the answer 0



The Problem

The Problem With Our First Attempt

```
void hello()
{
    printf("Hello\n");
    hello();
}
```

There is nothing telling the function to **stop.**



Therefore:

hello
↓
hello
↓
hello
↓
hello
↓
hello
↓
...





Key idea

A recursive function needs a way to **STOP.**

That stopping condition is called the:

BASE CASE



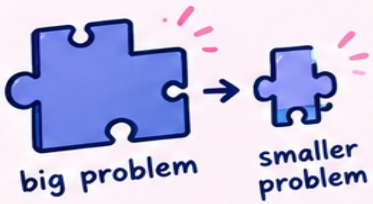
WHAT IS RECURSION?

Recursion = Same Problem, Smaller Problem

A powerful problem-solving pattern:

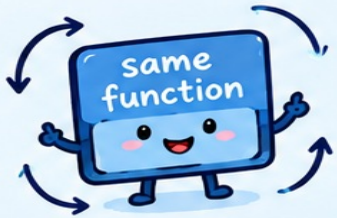
1 Break

Break the problem into a smaller version of itself.



2 Solve

Use the same function to solve the smaller problem.



3 Combine

Use that smaller solution to solve the original problem.



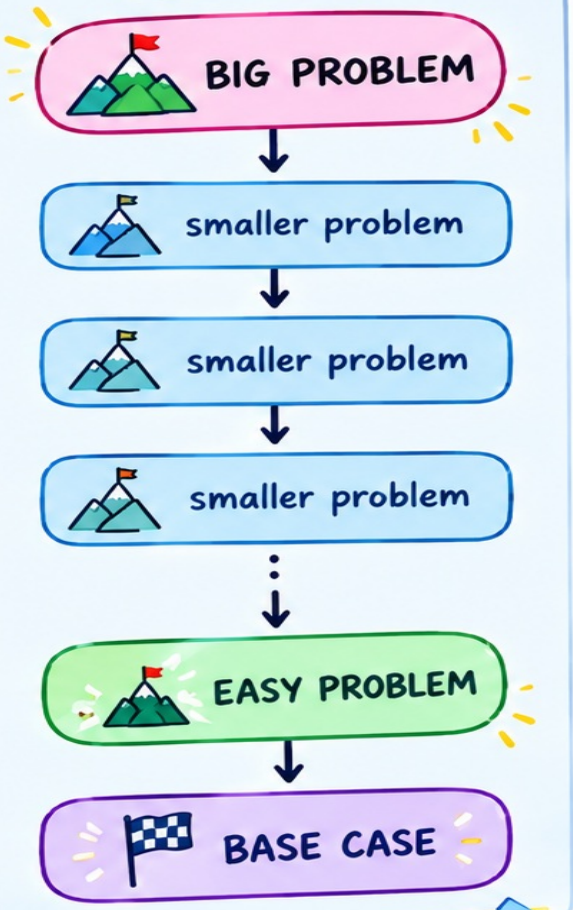
smaller solution + combine



It's like solving the same puzzle... again and again... but smaller each time!

Same logic.
Smaller problem.
Progress!

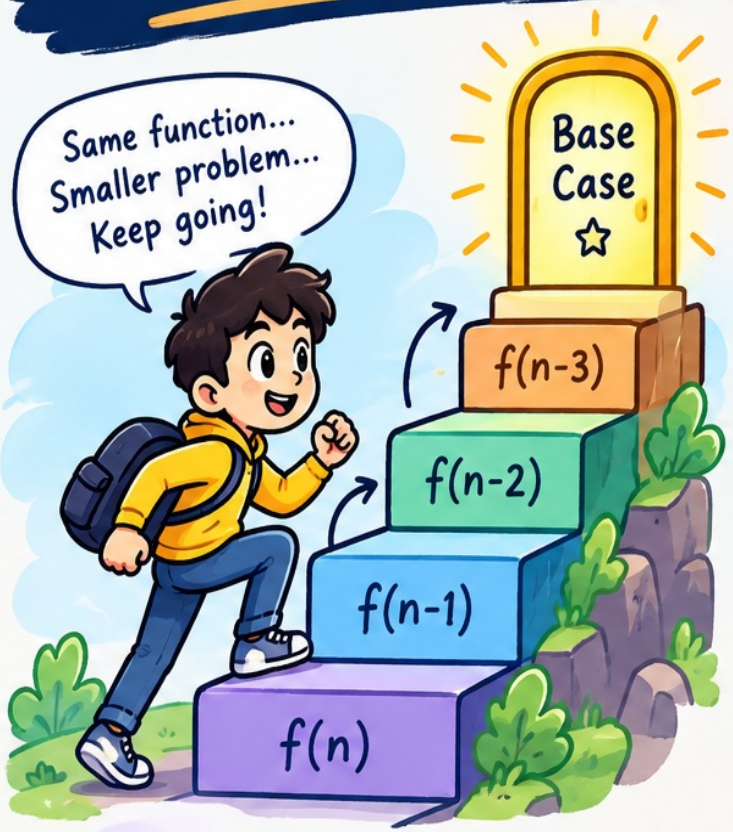
The Flow



Smaller and smaller... until it's easy!

Whenever you see a recursive problem, ask:

The Golden Pattern



1 What is the smaller problem?

f(n) → f(n-1)

(e.g., n → n-1)

2 Can I call the **SAME** function to solve it?

f(n) → f(n-1) — Same function!

3 What is the simplest case? → **BASE CASE**

f(0) — No more smaller problems! ☺

Remember:

Recursion is primarily a problem-solving technique.

Let's Meet Factorial!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

You already know: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

$$5! = \mathbf{5} \times 4!$$
$$4! = \mathbf{4} \times 3!$$
$$3! = \mathbf{3} \times 2!$$
$$2! = \mathbf{2} \times 1!$$
$$1! = \mathbf{1} \times 0!$$

The Base Case

$$0! = 1$$

This is where the recursion stops!

Every factorial reduces to a smaller factorial:

$$n! = n \times (n-1)!$$

Factorial Is Already Recursive!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

$$\boxed{n! = n \times (n-1)!}$$

Let's start with a big problem:

$5! \longrightarrow 4! \longrightarrow 3! \longrightarrow 2! \longrightarrow 1!$

5! becomes **4!**

4! becomes **3!**

3! becomes **2!**

2! becomes **1!**

The Big Idea

We keep making the problem smaller!

But Where Do We STOP?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

We keep making the problem smaller...

$$5! \rightarrow 4! \rightarrow 3! \rightarrow 2! \rightarrow 1! \rightarrow \boxed{0!}$$

STOP!

$$\boxed{0! = 1}$$

This is our BASE CASE.

FACTORIAL

$n = 0$

$\boxed{1}$

$n > 0$

$\boxed{n \times factorial(n-1)}$

Our First Recursive Function!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```
int factorial(int n)
{
   if (n == 0)
      return 1;

   else
      return n * factorial(n - 1);
}
```

$n = 0 \implies 1$

$n > 0 \implies n \times factorial(n-1)$

Read it in English

IF n **is** $0 \Rightarrow$ **answer is 1**

OTHERWISE $\Rightarrow$ **answer is** $n \times factorial(n-1)$

Let's Not Rush!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose we ask:

`factorial(5)`

`factorial(5)`

What is the smaller problem?

`factorial(4)`

What does `factorial(5)` do?

`5 * factorial(4)`

We **do not yet know** the value of `factorial(4)`.
So the function asks another copy of itself:

"Hey factorial(4), you figure that out!"

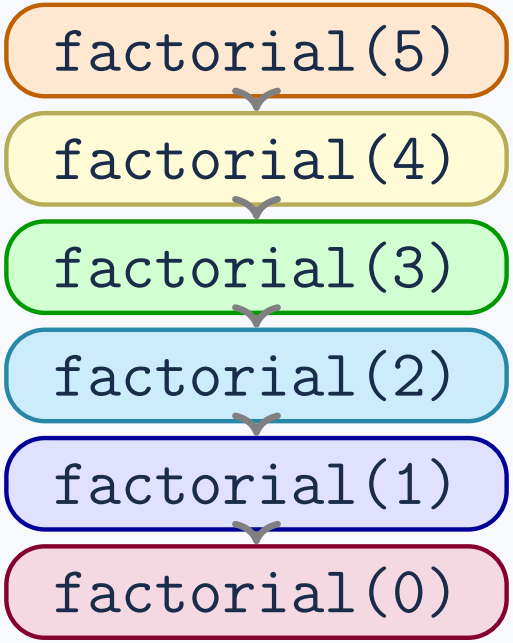
The Magic Behind Recursion

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

When a function calls another function, C remembers:

"I need to come back here later."

`factorial(5)`



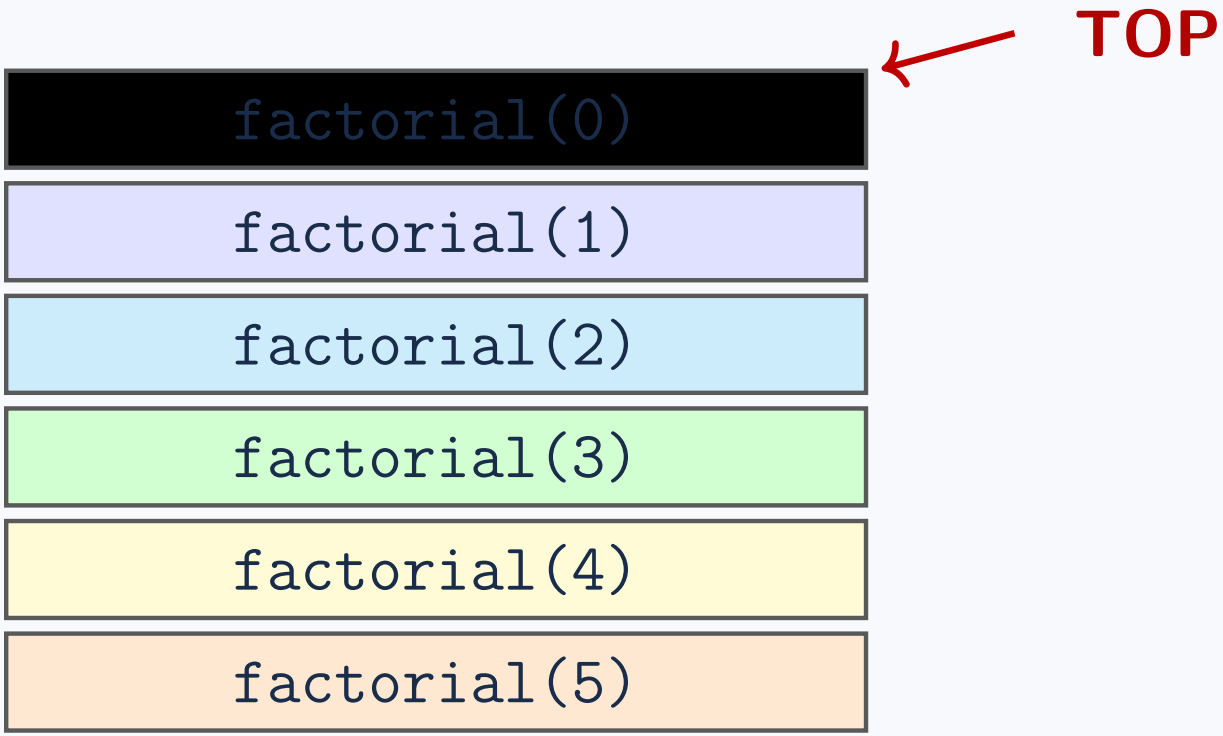
This is the CALL STACK

Function calls are stacked on top of each other.

Visualize the Stack

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Imagine the calls as a stack of books:



Every function call gets its own
STACK FRAME.

Each frame remembers what that particular call needs when it gets a chance to continue.

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What does this print?

```
int factorial(int n)
{
   if (n == 0)
      return 1;

   return n * factorial(n - 1);
}

int main()
{
   printf("%d", factorial(3));
}
```

A. 3

B. 6

C. 9

D. Infinite loop

What REALLY Happens?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Let's slowly follow:

$$\texttt{factorial(3)}$$

$$\texttt{factorial(3)} = 3 \times \texttt{factorial(2)}$$

$$= 3 \times [2 \times \texttt{factorial(1)}]$$

$$= 3 \times [2 \times [1 \times \texttt{factorial(0)}]]$$

$$\texttt{factorial(0)} = 1$$

Now the waiting calls can finally return!

$$3 \times 2 \times 1 \times 1 = \mathbf{6}$$

Go down to the base case → then come back up!

Going DOWN... Then Coming BACK UP!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

GOING DOWN ↓

`factorial(3)`
`factorial(2)`
`factorial(1)`
`factorial(0)`

BASE CASE FOUND!

$$\texttt{factorial}(0) = 1$$

$\texttt{factorial}(3) \rightarrow 3 \times 2 = 6$
$\texttt{factorial}(2) \rightarrow 2 \times 1 = 2$
$\texttt{factorial}(1) \rightarrow 1 \times 1 = 1$
$\texttt{factorial}(0) \rightarrow 1$

COMING BACK ↑

TRACE:Can you trace `factorial(4)`?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Call	What is waiting?
`factorial(4)`	$4\times$?
`factorial(3)`	$3\times$?
`factorial(2)`	$2\times$?
`factorial(1)`	$1\times$?
`factorial(0)`	$\boxed{1}$

`factorial(4)` = ?

`factorial(3)` = ?

`factorial(2)` = ?

`factorial(1)` = ?

`factorial(0)` = ?

Now go upward! ↑

1	1	2	6	24

Why Does Recursion Work?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Think about the smaller problem first.

Suppose we trust that `factorial(n-1)` already works correctly.

If

$$\boxed{\texttt{factorial(n-1)} = (n-1)!}$$

then the recursive function says:

$$\begin{aligned}\texttt{factorial(n)} &= n \times \texttt{factorial(n-1)} \\ &= n \times (n-1)! \\ &= \boxed{n!}\end{aligned}$$

The Recursive Leap

Solve the smaller problem → use it to solve the current problem.

Don't think:

"How will the computer execute every recursive call?"

Instead think:

"Assume the function already knows how to solve a smaller problem."

Then ask:

Can I use that answer to solve my current problem?

I trust `factorial(n-1)`

↓

It gives $(n-1)!$

↓

Multiply by n

↓

Get $n!$

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

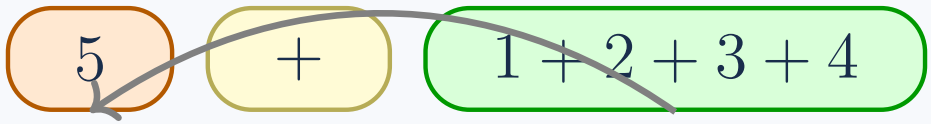
Suppose we want:

$$\boxed{sum(1, 2, 3, 4, 5) = 15}$$

Look at the sum in a different way:

$$1 + 2 + 3 + 4 + 5 = \boxed{5 + (1 + 2 + 3 + 4)}$$

same problem, but smaller



$$\boxed{sum(n) = n + sum(n - 1)}$$

Base Case

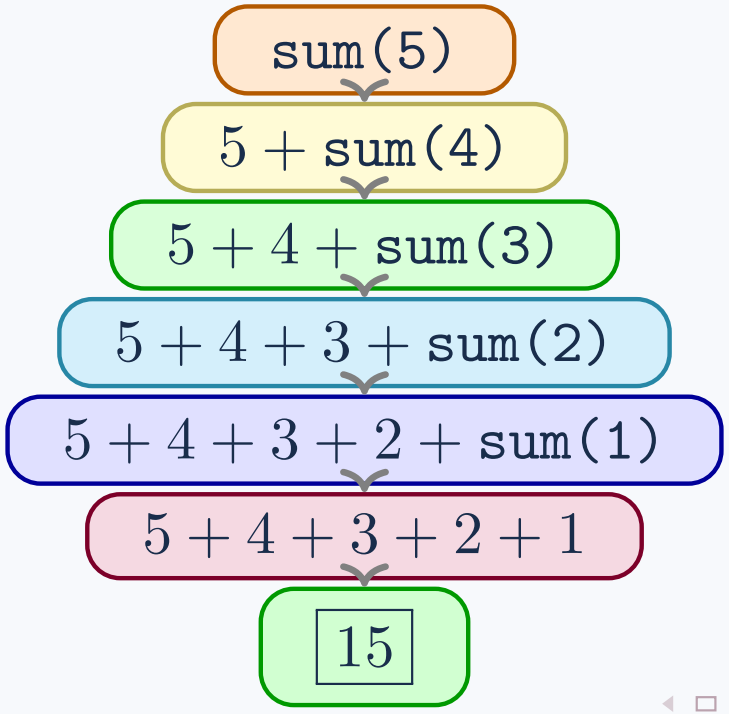
$$\boxed{sum(1) = 1}$$

C Code: Recursive Sum

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```
int sum(int n)
{
   if (n == 1)
      return 1;

   return n + sum(n - 1);
}
```



PREDICT THE OUTPUT

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What is printed?

```
int fun(int n)
{
   if (n == 0)
      return 0;

   return n + fun(n - 1);
}

int main()
{
   printf("%d", fun(4));
}
```

A. 4 | B. 6 | C. 10 | D. 24

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What does this print?

```
void fun(int n)
{
   if (n == 0)
      return;

   printf("%d ", n);
   fun(n - 1);
}
```

For: `fun(5);`

A. 1 2 3 4 5 **B. 5 4 3 2 1** **C. 5 4 3 2 1 0**

Look carefully!
`printf()` happens **BEFORE** the recursive call.

Reveal — Why 5 4 3 2 1?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

`fun(5)` → **print 5**

`fun(4)` → **print 4**

`fun(3)` → **print 3**

`fun(2)` → **print 2**

`fun(1)` → **print 1**

`fun(0)` → **STOP**

OUTPUT

5 4 3 2 1

Why? The printing happens while we are GOING DOWN.

Move ONE Line... and Everything Changes!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Version 1

```
void fun(int n)
{
   if (n == 0)
      return;

   printf("%d ", n);
   fun(n - 1);
}
```

5 4 3 2 1

Now move ONE line!

Move ONE Line... and Everything Changes!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Now move ONE line!

```
void fun(int n)
{
   if (n == 0)
      return;
   fun(n - 1);
   printf("%d ", n);
}
```

Same recursion...

But completely different output!

PREDICT AGAIN

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

For `fun(5);`, what happens?

A. 5 4 3 2 1

B. 1 2 3 4 5

C. 5 1 4 2 3

D. No

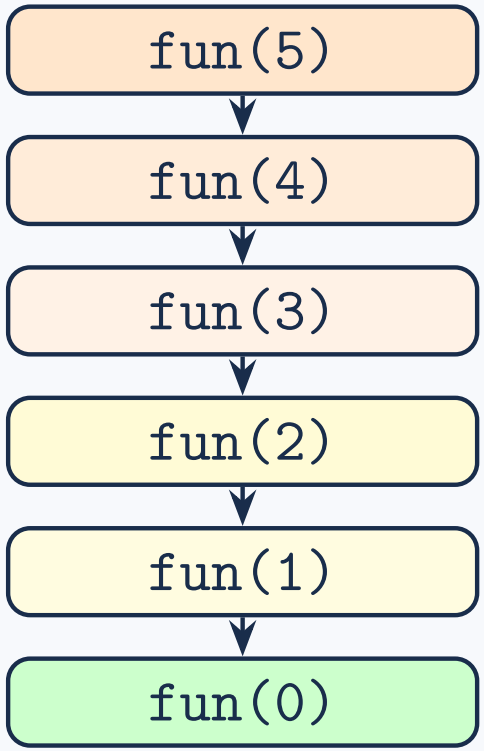
Think carefully!

The `printf()` is now **AFTER** the recursive call.

REVEAL! — The Answer Appears on the Way Back

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

The recursive call happens BEFORE printing!



GOING DOWN ↓

Base case reached!

COMING BACK UP ↑



Output

```
1 2 3 4 5
```



Today's Class (7/10/2026)

One Base Case Is Not Mandatory

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

A recursive function can have:

ONE base case

OR

MULTIPLE base cases

Example: Fibonacci Numbers

$$\boxed{F(0) = 0} \qquad \boxed{F(1) = 1}$$

These are **two base cases**.
For larger values:

$$\boxed{F(n) = F(n-1) + F(n-2)}$$

Why does Fibonacci need TWO stopping points?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Fibonacci has TWO base cases!

$F(0) = 0$

$F(1) = 1$

```
int fib(int n)
{
if (n == 0)
return 0;

if (n == 1)
    return 1;

return fib(n - 1) + fib(n - 2);

}
```

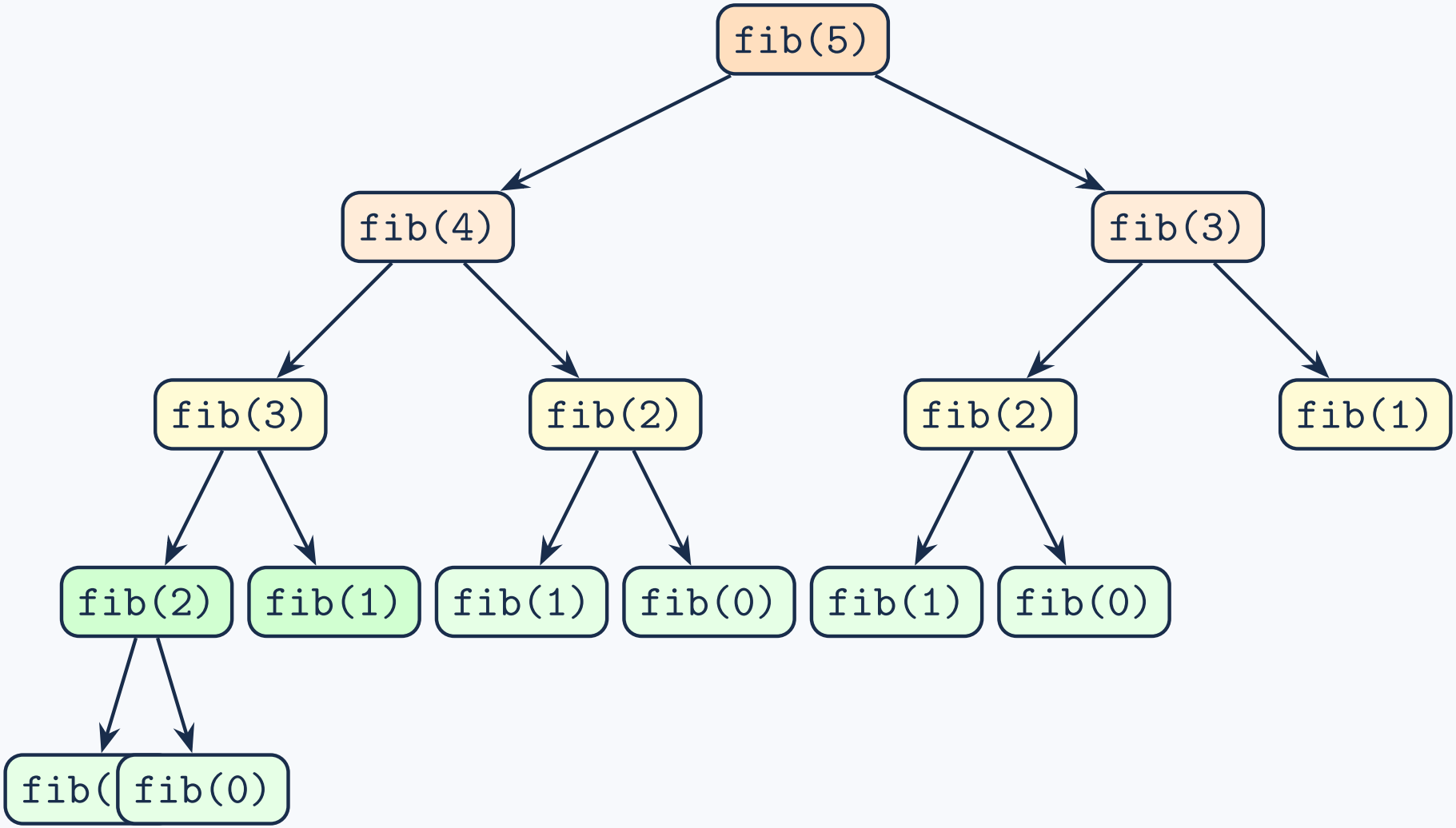
Notice the Recursive Step

$$F(n) = F(n-1) + F(n-2)$$

What Happens When We Call `fib(5)`?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Watch the single call split into two!



One call → Two smaller calls → A RECURSION TREE!

New Data Type: Structures

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

Today: Meet a New Data Type



Imagine we want to store information about a student.

Name	→	"Rahul"
Roll No.	→	23
CGPA	→	8.7
Age	→	18

Question:

Can we store all of this in **ONE** variable?



They have DIFFERENT data types!

name	roll	cgpa	age
→ character array	→ int	→ float	→ int
`char name[20];`	`int roll;`	`float cgpa;`	`int age;`



Today we learn a way to package them together. → **STRUCTURES**

Structures

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

First, What Problem Are We Solving?

Suppose we have:

```
char name[50];
int roll;
float cgpa;
int age;
```

These variables describe **ONE** student.



But C sees them as four completely **unrelated** variables.

name	→	"Rahul"
roll	→	23
cgpa	→	8.7
age	→	18

There is no explicit connection saying:

"These four things belong to the **SAME** student."





We want:

STUDENT

name	char[50]
roll	int
cgpa	float
age	int

All information about **one student** is packed together in a single variable.

That is exactly what a **struct** does.



In short: A struct lets us group related data together in one variable.



Structures

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

Real-Life Analogy

Think about a house.



A house has:

House

- number
- owner name
- area
- number of rooms
- price

These properties have different types.

number	→	int
owner name	→	char[]
area	→	float
rooms	→	int
price	→	float

But together they describe **ONE** house.





Similarly:

A student has:

Student

- name
- roll
- age
- cgpa

These properties are related and describe **ONE** student.





A structure lets us create our own **custom data type.**

Like a house or a student — but in code!

Structures

॥ सा विद्या या विमुक्तये ॥
य प्रौद्योगिकी संस्थान धारवाड
itute of Technology Dharwad

What Is a Structure?

A structure is a collection of variables of **different data types** grouped together under **one name.**

In C:

```
1  struct Student
2  {
3      char name[50];
4      int roll;
5      float cgpa;
6      int age;
7  };
```

Think:

struct Student

↓

A new **blueprint/type.**

Student

name	(char[50])
roll	(int)
cgpa	(float)
age	(int)



Important:

- ✓ This creates a structure type.
- ✓ It does **NOT** yet create a student variable.

Structures

Structure = Blueprint

Very important distinction.

```
struct Student
{
    char name[50];
    int roll;
    float cgpa;
};
```

This is like a blueprint.



BLUEPRINT

↓

`struct Student`

↓

Student

name	(char[50])
roll	(int)
cgpa	(float)

→

Now we can create actual students.

`struct Student s1;`

s1

name	""
roll	0
cgpa	0.0



Now s1 is an actual variable.

Structures

Structure Variable

```
struct Student s1;
```

Now memory contains something like:

Memory View

s1

name	"Rahul"
roll	23
cgpa	8.7
...	

We can store:

```
s1.roll = 23;
s1.cgpa = 8.7;
```

? But how do we access individual fields?

Meet the:

DOT (.) operator



This lets us access the individual fields of a structure variable.

Structures

The DOT Operator

To access a structure member:

structure_variable.member

Example:

`s1.roll`

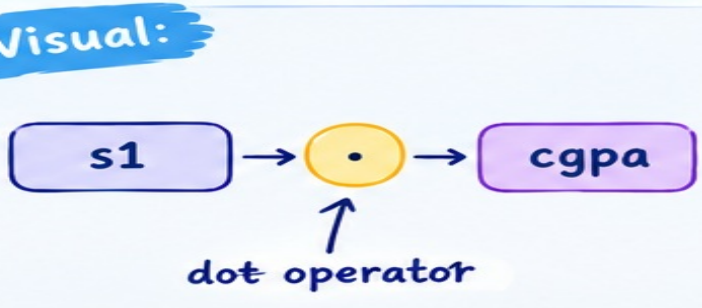
means:

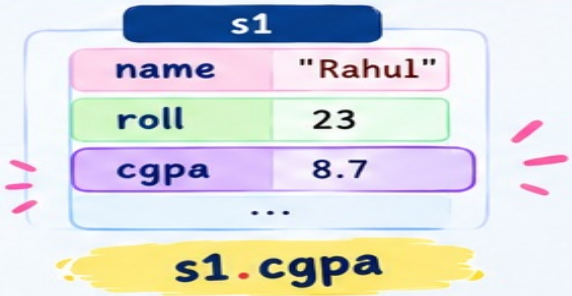
"Give me the roll field belonging to s1."

Similarly:

s1.cgpa	(float)
s1.age	(int)
s1.name	(char[])

Visual:







In short: The dot (.) operator helps us access individual fields of a structure variable.



॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Step 1: Define the Student

```
struct Student
{
    char name[50];
    int roll;
    float cgpa;
};
```

⇓ Step 2: Create one Student

```
struct Student s1;
```

⇓

```
s1.roll = 23;
s1.cgpa = 8.7;
```

What happened?

`s1` is one Student object containing its own `name`, `roll`, and cgpa.

PREDICT THE OUTPUT



```
struct Student
{
    int roll;
    float cgpa;
};
int main()
{
    struct Student s;
    s.roll = 10;
    s.cgpa = 9.2;
    s.roll = 25;
    printf("%d %.1f", s.roll, s.cgpa);
}
```

A. 10 9.2 **B.** 25 9.2 **C.** 25 0 **D.** Error

PREDICT THE OUTPUT



```
struct Student
{
    int roll;
    float cgpa;
};
int main()
{
    struct Student s;
    s.roll = 10;
    s.cgpa = 9.2;
    s.roll = 25;
    printf("%d %.1f", s.roll, s.cgpa);
}
```

A. 10 9.2 **B.** 25 9.2 **C.** 25 0 **D.** Error

Answer: B — 25 9.2

`s.roll = 25;` changes **only the roll member**.

Structure Members Behave Like Normal Variables

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Once we access a member...

```
s.roll
```

it behaves like a normal `int` variable.

Assign

```
s.roll = 25;
```

Print

```
printf("%d", s.roll);
```

Even input is possible!

```
scanf("%d", &s.roll);
```

Similarly, `s.cgpa` behaves like a normal `float`.

Reading a Structure from the User

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Let's take input!

```
struct Student s;
printf("Enter roll: ");
scanf("%d", &s.roll);

printf("Enter CGPA: ");
scanf("%f", &s.cgpa);
```

Why `&s.roll`?

`scanf()` needs the **address**
where it should store the value.

Same idea as:

`int x;` $\longrightarrow$ `scanf("%d", &x);`

$\Downarrow$

Only the variable is different!



Today's Class (9/10/2026)

What About Strings? `char name[50]`

One Interesting Difference!

```
struct Student
{
    char name[50];
    int roll;
};
```

We can take the name using:

```
scanf("%s", s.name);
```

Notice: NO &!

```
scanf("%d", &s.roll);
```

↑ needs &

```
scanf("%s", s.name);
```

↑ does NOT need &

Why?

`s.name` is an array.
An array name already behaves like an address in this context.

Initialization at Declaration

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

We can initialize a structure immediately!

```
struct Student s1 = {"Rahul", 23, 8.7};
```

The values are assigned **in member order**:

`name` $\longrightarrow$ `"Rahul"`

`roll` $\longrightarrow$ `23`

`cgpa` $\longrightarrow$ `8.7`

This is similar to initializing an array:

```
int a[3] = {10, 20, 30};
```

Values are filled from left to right.

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose we want to represent a BOOK

What information do we need?

A Book has...

title | author | price | pages

What data type should each member have?

`title`	`author`	`price`	`pages`
?	?	?	?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Suppose we want to represent a BOOK

What information do we need?

A Book has...

title | author | price | pages

What data type should each member have?

title	author	price	pages
?	?	?	?

One Possible Design

```
struct Book
{
    char title[100];
    char author[50];
    float price;
    int pages;
};
```

Challenge: Design a Movie Structure

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Design a structure to represent a **Movie**.

A Movie has...

name **year** **rating** **duration**

Choose the appropriate data type for each:

`name`	`year`	`rating`	`duration`
?	?	?	?

Challenge: Design a Movie Structure

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Design a structure to represent a **Movie**.

A Movie has...

name **year** **rating** **duration**

Choose the appropriate data type for each:

`name`	`year`	`rating`	`duration`
?	?	?	?

Possible Solution

```
struct Movie
{
    char name[100];
    int year;
    float rating;
    int duration;
};
```

Notice: One structure can contain **different data**

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

An Excellent Question!

Suppose we store 100 students using separate arrays:

Separate Arrays

```
int roll[100];    float cgpa[100];    char name[100][50];
```

	Roll	CGPA	Name
Student 0	`roll[0]`	`cgpa[0]`	`name[0]`
Student 1	`roll[1]`	`cgpa[1]`	`name[1]`
Student 2	`roll[2]`	`cgpa[2]`	`name[2]`

The Problem

We must manually maintain the relationship:

$$\texttt{roll[0]} \leftrightarrow \texttt{cgpa[0]} \leftrightarrow \texttt{name[0]}$$

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

An Excellent Question!

Suppose we store 100 students using separate arrays:

Separate Arrays

```
int roll[100];    float cgpa[100];    char name[100][50];
```

	Roll	CGPA	Name
Student 0	`roll[0]`	`cgpa[0]`	`name[0]`
Student 1	`roll[1]`	`cgpa[1]`	`name[1]`
Student 2	`roll[2]`	`cgpa[2]`	`name[2]`

The Problem

We must manually maintain the relationship:

`roll[0]` $\leftrightarrow$ `cgpa[0]` $\leftrightarrow$ `name[0]`

Structures give us a better idea!

```
struct Student students[100];
```

Why Not Just Use Arrays?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Structures give us a better idea!

```
struct Student students[100];
```

Now:

`students[0]` `students[1]` `students[2]`

Each element represents a **COMPLETE STUDENT**

⇓

ARRAY OF STRUCTURES

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

One Student → Many Students

```
struct Student
{
    char name[50];
    int roll;
    float cgpa;
};
```

Now create an array:

```
struct Student students[3];
```

Think of it as **3 complete Student structures**:

Array Element	name	roll	cgpa
students[0]	—	—	—
students[1]	—	—	—
students[2]	—	—	—

Key Idea

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

How do we reach a member?

`students[0].roll`

Read it from **left to right**:

`students`	Array
↓	
`students[0]`	First Student
↓	
`students[0].roll`	Roll of first Student

Similarly:

`students[1].cgpa`	`students[2].name`
Second student's CGPA	Third student's name

Remember

`[ ]` selects the **structure** `.` selects the **member**

PREDICT!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```
struct Student
{
    int roll;
    float cgpa;
};
struct Student s[3];
s[0].roll = 10;
s[1].roll = 20;
s[2].roll = 30;
printf("%d", s[1].roll);
```

A. 10 **B.** 20 **C.** 30 **D.** Error

PREDICT!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```
struct Student
{
    int roll;
    float cgpa;
};
struct Student s[3];
s[0].roll = 10;
s[1].roll = 20;
s[2].roll = 30;
printf("%d", s[1].roll);
```

A. 10 **B.** 20 **C.** 30 **D.** Error

Answer: B — 20

`s[1]` means the **second structure**.

Therefore `s[1].roll` is **20**.

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Process Many Students!

```
for(int i = 0; i < 3; i++)
{
    printf("Enter roll: ");
    scanf("%d", &students[i].roll);
    printf("Enter CGPA: ");
    scanf("%f", &students[i].cgpa);
}
```

Then print all students:

```
for(int i = 0; i < 3; i++)
{
    printf("%d %.2f\n", students[i].roll, students[i].cgpa);
}
```

The Powerful Pattern

`i` selects the student + `.member` selects the information

One loop → many complete students!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Your Turn!

Write a program to store information of **3 students**.

Each Student has:

Name **Roll Number** **CGPA**

Your program should finally print:

```
Student 1: Rahul 23 8.7
Student 2: Priya 17 9.1
Student 3: Aman 31 7.8
```

Searching in an Array of Structures

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose we have:

```
struct Student students[100];
```

Find the student whose **roll number is 25**.

Linear Search

```
int target = 25;
for(int i = 0; i < 100; i++)
{
    if(students[i].roll == target)
    {
        printf("Found!");
        break;
    }}
```

`students[i].roll` $\longrightarrow$ **the roll number of student** `i`

This is just **linear search**. The only difference: **each array element is a complete object.**

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Can We Sort Students?

Yes!

For example, sort students by **CGPA**.

```
if(students[i].cgpa > students[j].cgpa)
{
    // swap students[i] and students[j]
}
```

But what exactly should we swap?

`students[i].cgpa` × `students[j].cgpa`

Not just the CGPA!

Swap the **ENTIRE STRUCTURES**!

`students[i]` $\Longleftrightarrow$ `students[j]`

Name + Roll + CGPA all move together.

Today's Class (10/10/2026)

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Can We Copy One Structure to Another?

```
struct Student s1;
struct Student s2;
```

Can we write:

```
s2 = s1;
```

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Can We Copy One Structure to Another?

```
struct Student s1;
struct Student s2;
```

Can we write:

```
s2 = s1;
```

YES!

The structure's members are copied.



Remember

Structures can be assigned directly.

But ordinary arrays cannot be assigned using =.

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Can We Compare Two Structures?

Suppose:

```
struct Student s1;
struct Student s2;
```

Can we write?

```
if(s1 == s2)
```

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Can We Compare Two Structures?

Suppose:

```
struct Student s1;
struct Student s2;
```

Can we write?

```
if(s1 == s2)
```

NO!

Compare the members instead

```
if(s1.roll == s2.roll &&
   s1.cgpa == s2.cgpa)
{
    ...
}
```

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What about names?

Correct	Not this
`strcmp(s1.name, s2.name)`	`s1.name == s2.name`

Remember

Structure comparison = compare the relevant members.

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Where Does a Structure Live?

Consider:

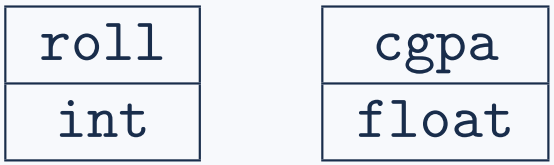
```
struct Student
{
    int roll;
    float cgpa;
};
```

Now create one:

```
struct Student s;
```

What happened?

s occupies a region of **memory**.



How much memory does it occupy?

```
sizeof(s)     or     sizeof(struct Student)
```

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

How much memory does a structure occupy?

We can ask C directly:

```
printf("%zu", sizeof(struct Student));
```

The size depends on its members.

```
struct Student
{
    int roll;
    float cgpa;
};
```

At first, we may think:

$$\boxed{\text{size of structure} = \text{size of int} + \text{size of float}}$$

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

How much memory does a structure occupy?

We can ask C directly:

```
printf("%zu", sizeof(struct Student));
```

The size depends on its members.

```
struct Student
{
    int roll;
    float cgpa;
};
```

At first, we may think:

$$\boxed{\text{size of structure} = \text{size of int} + \text{size of float}}$$

But be careful!

C may insert extra bytes called **padding**
for proper memory alignment.

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

A Structure Has an Address!

Suppose:

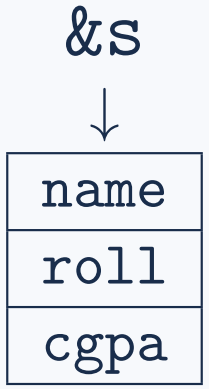
```
struct Student s;
```

We can get the address of the **entire structure**:

```
&s
```

Just like an ordinary variable:

```
int x;
&x
```



Entire structure

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

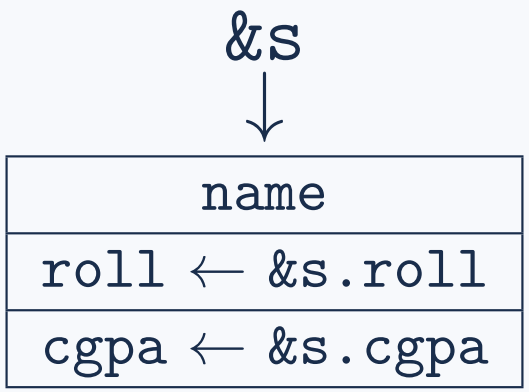
We Can Also Get Member Addresses!

Suppose:

```
struct Student s;
```

We can ask for:

`&s.roll` `&s.cgpa`



Remember

`&s` → address of the **entire structure** `&s.roll` → address of the **roll member**

`&s.cgpa` → address of the **CGPA member**

A Structure Is NOT an Array

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Don't Confuse Them!

ARRAY

```
int a[3];
```

int	int	int

All elements have the **same type**.

STRUCTURE

```
struct Student

{
 char name[50];
 int  roll;
 float cgpa;
};
```

Members can have **different types**.

ARRAY $\longrightarrow$ **STRUCTURE**

Collection of $\longrightarrow$ Collection of

similar data $\longrightarrow$ **related data**

Array = similar data

Structure = related data

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Can We Send a Structure to a Function?

YES! ✓

First, define our structure:

```
struct Student
{
    int roll;
    float cgpa;
};
```

Now define a function:

```
void printStudent(struct Student s)
{
    printf("%d %.2f",
           s.roll,
           s.cgpa);
}
```

Passing a Structure to a Function

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Call the function:

```
printStudent(s1);
```

A complete structure is passed to the function.

Think About What Is Happening

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What happens here?

```
printStudent(s1);
```

main()
s1
roll
cgpa

COPY →

printStudent()
s
roll
cgpa

Key Idea

The function receives a **copy** of the structure.
This is called **pass by value**.

Can the Function Modify the Original?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

PREDICT THE RESULT!

Suppose:

```
void change(struct Student s)
{
    s.roll = 100;
}
```

and:

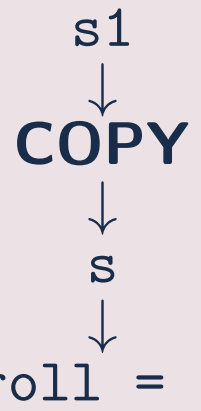
```
change(s1);
```

Will `s1.roll` become 100?

Can the Function Modify the Original?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Answer: B — NO



The original `s1` remains unchanged.

Remember

Pass by value → function works on a copy.

Want to Modify the Original?

What if a function should change the original structure?

Remember

Earlier, we passed the structure **by value**: `change(s1);`
The function received a **copy**.

Solution: Pass the Address

```
void change(struct Student *s)
{
    s->roll = 100;
}
```

Call the function using:

```
change(&s1);
```

Key Idea

The function now receives a **pointer to the original structure**. So it can modify the original `s1`.

How do we access a member through a pointer?

Suppose we have the actual structure:

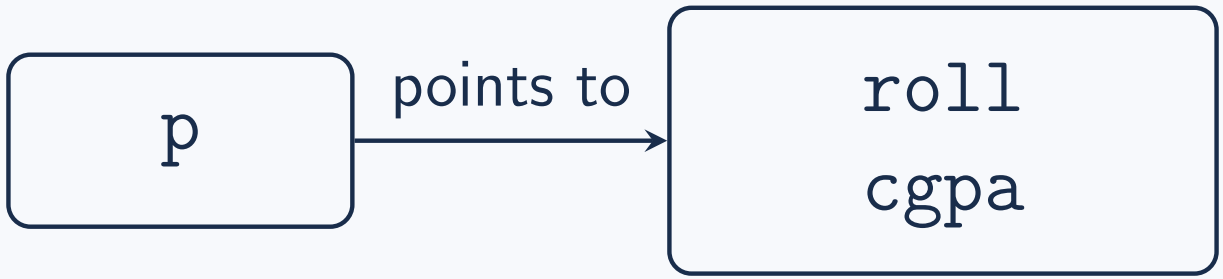
```
struct Student s1;
```

Then: `s1.roll`

But suppose we have a pointer:

```
struct Student *p;
```

Now use: `p->roll`



Meaning

`p->roll` means: **Go to the structure pointed to by `p`, then access its `roll`.**

The most important rule to remember

Have the structure?

Use .

```
struct Student s;

s.roll
s.cgpa
```

Have a pointer?

Use ->

```
struct Student *p;

p->roll
p->cgpa
```

`s.roll` **VS** `p->roll`

Easy Memory Trick

Structure → . **Pointer to Structure → ->**

What is really happening?

We write:

```
p->roll
```

But internally, it means:

```
(*p).roll
```

Why the parentheses?

Because `.` has higher precedence than `*`.

Without parentheses:

```
*p.roll
```

would be interpreted as:

```
*(p.roll)
```

which is **not what we want**.

The Secret Behind ->

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Remember

```
p->roll   =   (*p).roll
```

. $\rightarrow$ actual structure -> $\rightarrow$ pointer to structure

PREDICT THE OUTPUT

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What will be printed?

```
struct Student
{
    int roll;
};
void change(struct Student *p)
{
    p->roll = 99;
}
int main()
{
    struct Student s;
    s.roll = 10;
    change(&s);
    printf("%d", s.roll);
}
```

A. 10 B. 99 C. Error D. Garbage

PREDICT THE OUTPUT

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Think Carefully!

What does `&s` give to the function?

Answer

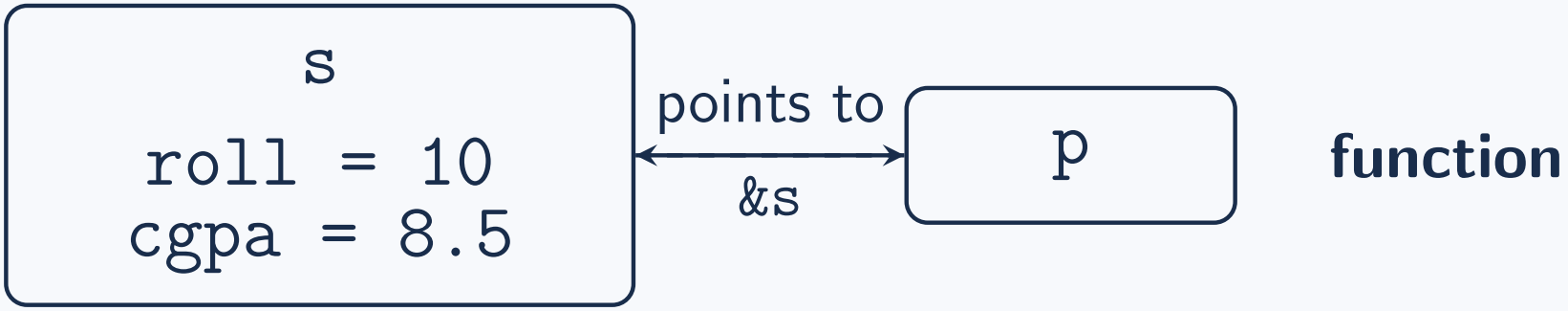
B. 99

The function receives the **address of** `s`.
So `p->roll = 99` changes the original `s.roll`.

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

This is what is happening in memory

main()



Inside the function

```
p->roll = 99;
```

⇓

```
s.roll = 99;
```

p does not contain a new Student. p contains the **address of the original Student**.

Passing a Structure to a Function: Two Ways

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

There are two common ways

Method 1 — Pass Structure

```
void print(
    struct Student s
)
```

Call:

```
print(s1);
```

Structure is passed as a **copy**.

`s1` ⟶ COPY

Method 2 — Pass Address

```
void update(
    struct Student *s
)
```

Call:

```
update(&s1);
```

Function receives the **address**.

`&s1` ⟶ ADDRESS

Remember

Pass structure → function gets a copy

Pass pointer → function can access the original

Why Pass a Pointer?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What if the structure is very large?

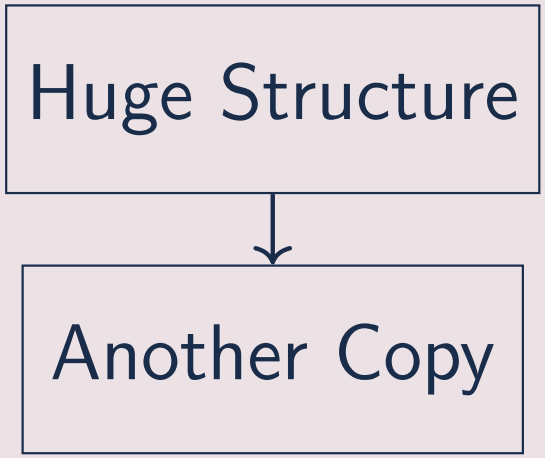
A Huge Structure

```
struct Student

    name
    roll
    cgpa
    age
    ...
many more members
```

Pass by Value

A **copy** is made.



Instead, pass only its address:

```
struct Student *p
```

Key Idea

A pointer stores an **address**, not a complete copy of the structure.

Why Pass a Pointer?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Large structure $\longrightarrow$ **Pointer** $\longrightarrow$ **Address**

Passing a pointer can avoid copying a large structure when a copy is not needed.

A Complete Example

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```
#include <stdio.h>
struct Student
{
    char name[50];
    int roll;
    float cgpa;
};
void printStudent(struct Student *s)
{
    printf("Name: %s\n", s->name);
    printf("Roll: %d\n", s->roll);
    printf("CGPA: %.2f\n", s->cgpa);
}
int main(){
    struct Student s;
    printf("Enter name: ");    scanf("%49s", s.name);
    printf("Enter roll: ");    scanf("%d", &s.roll);
    printf("Enter CGPA: ");    scanf("%f", &s.cgpa);
    printStudent(&s);    return 0;
}
```

A Complete Example

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Trace the Important Parts

`s.roll` $\rightarrow$ actual structure `&s` $\rightarrow$ address

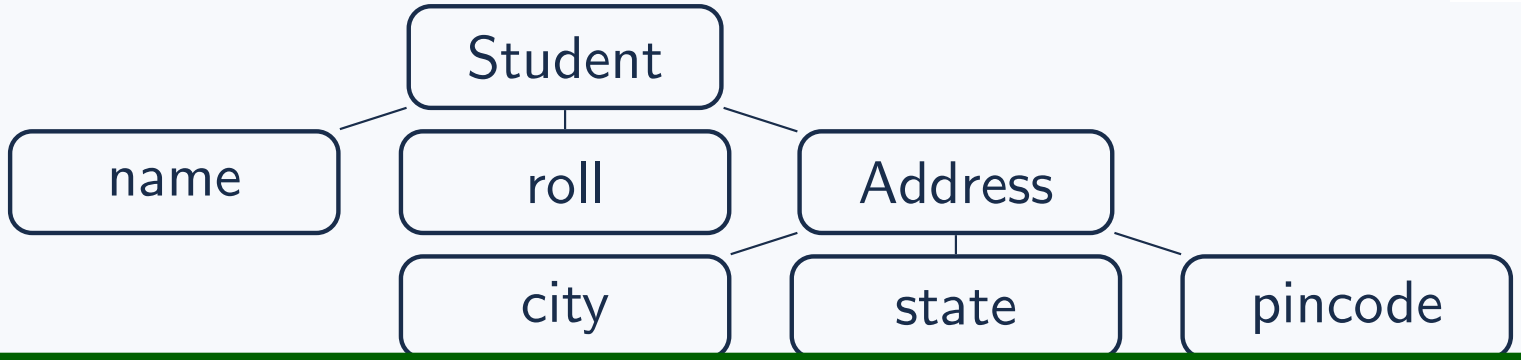
`struct Student *s` $\rightarrow$ pointer

`s->roll` $\rightarrow$ member through pointer

Structure Inside Structure

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Real-world objects can contain other objects!



C can represent this!

```
struct Address
{
    char city[50];
    char state[50];
    int pincode;
};
struct Student
{
    char name[50];
    int roll;
    struct Address address;
};
```

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose we create:

```
struct Student s;
```

To access the pincode:

`s.address.pincode`



`s` → `address` → `pincode`

Read it from left to right

Go to the student → go to the address → get the pincode

Structure + Array + Function =



1. Define a Structure

```
struct Student{
    char name[50];
    int roll;
    float cgpa;    };
```

2. Create an Array of Structures

```
struct Student students[100];
```

3. Create a Function

```
void printStudent(struct Student *s){
          printf("%s %d %.2f\n",
          s->name,
          s->roll,
          s->cgpa);}
```

4. Call the Function

```
printStudent(&students[i]);
```

STRUCTURE + **ARRAY** + **POINTER** + **FUNCTION**

A Very Useful Mental Model

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Translate the C code into simple English!

Case 1 — Array of Structures

`students[i].roll`

Say:

**"Go to the `i`th student,
then get the roll."**

Case 2 — Structure Pointer

`p->roll`

Say:

**"Follow pointer `p` to the student,
then get the roll."**

Can you spot the mistakes?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Mistake 1

`Student s;`

Not valid in basic C
(unless `typedef` is used).

Correct:

`struct Student s;`

Mistake 2

If `s` is an actual structure:

`s->roll`

Wrong! Correct:

`s.roll`

Mistake 3

If `p` is a structure pointer:

`p.roll`

Wrong!
Correct:

`p->roll`

Remember

Structure $\longrightarrow$.

Pointer $\longrightarrow$ ->

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Remember what `scanf()` needs!

Given:

```
struct Student
{
    int roll;
}; struct Student s;
```

Wrong

```
scanf("%d", s.roll);
```

Why? `scanf()` needs the **address** where it should store the value.

Correct

```
scanf("%d", &s.roll);
```

`s.roll` $\rightarrow$ the value

`&s.roll` $\rightarrow$ the address of the value

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What is the fundamental difference?

Feature	**Array**	**Structure**
Data types	Same	Can be different
Example	`int a[5]`	`struct Student`
Access	`a[i]`	`s.roll`
Main purpose	Many similar values	Related information
Can make an array?	Yes	Yes!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Array of Structures

```
struct Student students[100];
```

Think of it this way

Array $\rightarrow$ many things of the same type

Structure $\rightarrow$ different information about one thing

Array of Structures $\rightarrow$ many complete things

CHALLENGE — Build a Cricket Team



Step 1: Design a Player

Each player has three pieces of information.

```
struct Player
{
    char name[50];
    int runs;
    int wickets;
};
```

Step 2: Create a Team of 5 Players

```
struct Player team[5];
```

Your Task

1. Input the name, runs, and wickets of all 5 players.
2. Print the details of all players.
3. Find the player who scored the **maximum runs**.
4. Find the player who took the **maximum wickets**.

Debugging Challenge — Find the Bug!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

This program contains a mistake. Can you find it?

```
struct Student
{
    int roll;
    float cgpa;
};
void change(struct Student s)
{
    s.roll = 100;
}
int main()
{
    struct Student s;
    s.roll = 10;
    change(&s);
    printf("%d", s.roll);
}
```

The function expects:

```
struct Student
```

But the call sends:

```
&s
```

And `&s` has type:

```
struct Student *
```

The Problem

Type mismatch!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

How can we fix it?

Fix 1 — Pass the Structure

```
change(s);
```

The function receives a copy. The original `s.roll` remains 10.

Fix 2 — Pass the Address

```
void change(struct Student *s)
{
    s->roll = 100;
}
/* Call it using: */
change(&s);
```

The function receives a pointer and can modify the original structure.

MCQ 1 — Structure or Pointer?



```c
#include <stdio.h>
struct Point {
    int x;
    int y;
};
int main(void) {
    struct Point p = {10, 20};
    struct Point *q = &p;
    printf("%d ", q->x);
    printf("%d ", (*q).y);
    printf("%d", p.x);
    return 0;
}
```

What will this program print?

A. 10 20 10 [B.] 10 10 10

C. 20 20 10 [D.] Compilation error: q->x is invalid

MCQ 1 — Answer

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Correct Answer: A. 10 20 10

Step 1 — Understand the Structure

```
struct Point p = {10, 20};
```

Member	Value
`p.x`	10
`p.y`	20

Step 2 — Follow Each Print Statement

Expression	Value	Why?
`q->x`	10	Pointer accesses `x`
`(*q).y`	20	Dereference, then access `y`
`p.x`	10	Direct member access

MCQ 2 — Structure Passed by Value

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```c
#include <stdio.h>
struct Box {
    int value;
};
void modify(struct Box b) {
    b.value += 10;
}
int main(void) {
    struct Box b = {5};
    modify(b);
    printf("%d\n", b.value);
    return 0;
}
```

PREDICT THE OUTPUT!

A. 5 [B.]15

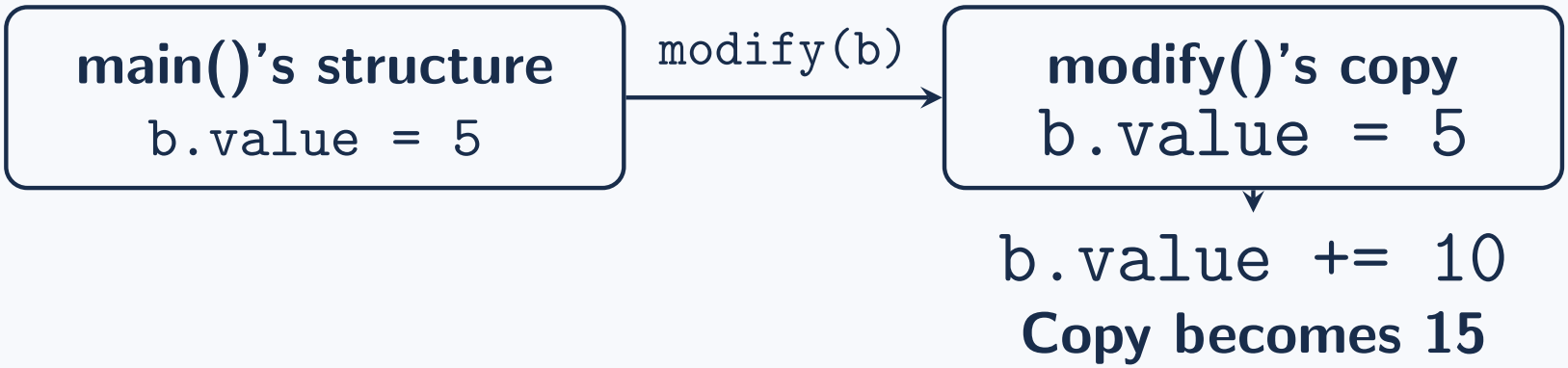
C. Garbage value [D.] Compilation error

MCQ 2 — Answer!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

A. 5

Why isn't the answer 15?



Location	Value
Before function call	5
Inside `modify()` after addition	15
Back in `main()`	5

Pass by value = the function receives a copy.
Changing the copy does not change the original structure.

MCQ 3 — Passing a Pointer to a Structure

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```c
#include <stdio.h>
struct Box {
    int value;
};
void modify(struct Box *b) {
    b->value += 10;
}
int main(void) {
    struct Box b = {5};
    modify(&b);
    printf("%d\n", b.value);
    return 0;
}
```

PREDICT THE OUTPUT!

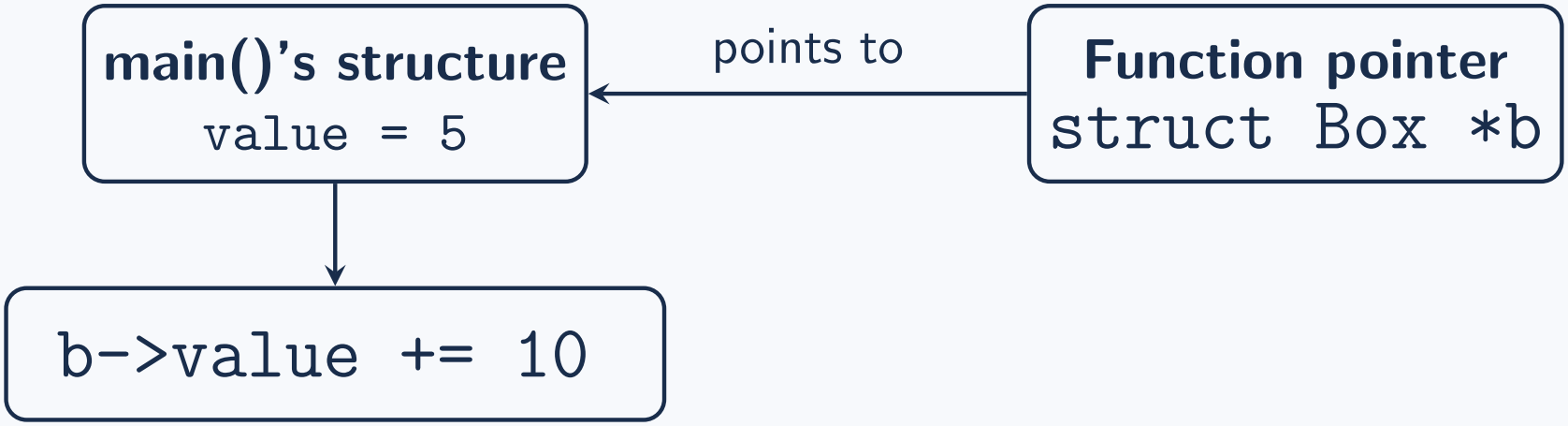
A. 5 [B.] 15

C. The program does not compile [D.] An unpredictable value

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

B. 15

Follow the address!



Original value becomes 15!

Step-by-Step

1. `b.value` starts at 5.
2. `modify(&b)` passes the address of the original structure.
3. `b->value += 10` changes the original member to 15.
4. Back in `main()`, `b.value` is still 15.

MCQ 4 — Array of Structures and Indexing

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```c
#include <stdio.h>
struct Item {
    int value;
};
int main(void) {
    struct Item a[4] = {{2}, {4}, {6}, {8}};
    struct Item *p = a;
    printf("%d\n", (p + 2)->value);
    return 0;
}
```

PREDICT THE OUTPUT!

A. 2

B. 4

C. 6

D. 8

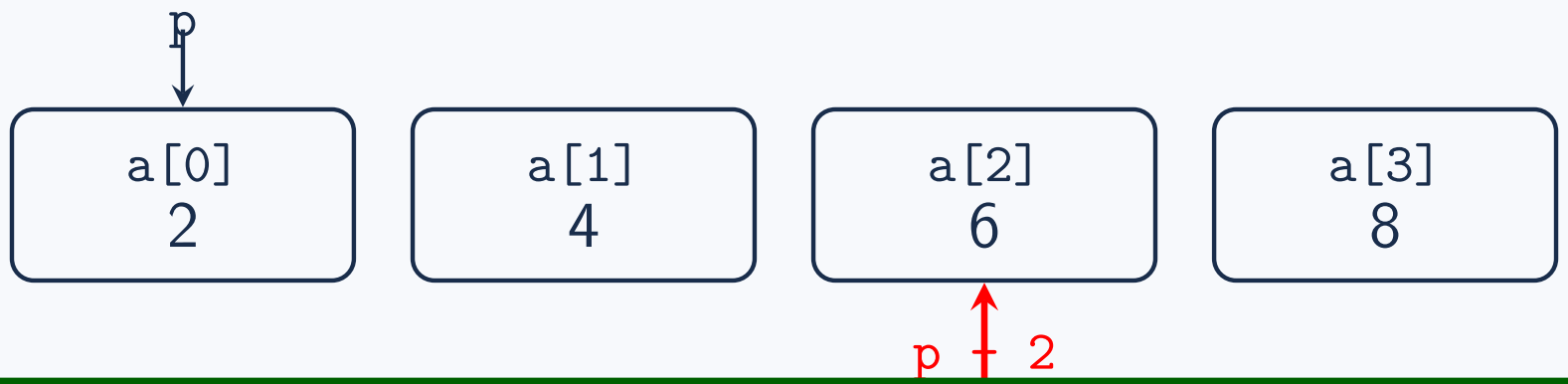
MCQ 4 — Follow the Pointer!

C. 6

Step 1 — Understand the Array

```
struct Item a[4] = {{2}, {4}, {6}, {8}};
struct Item *p = a;
```

`p = a` means `p` points to `a[0]`.



Step 2 — Follow the Expression

```
(p + 2)->value
```

1. `p + 2` points to the third element, `a[2]`.
2. `->value` accesses that structure's `value`.
3. The value stored there is 6.

MCQ 5 — Structure Size and Padding

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Consider the following structure:

```
struct Example {
    char c;
    int  x;
    char d;
};
```

What can we correctly say about `sizeof(struct Example)`?

Choose the Correct Answer

A. It is always 6 bytes.

B. It is always 3 bytes.

C. It depends on the implementation, including alignment and padding.

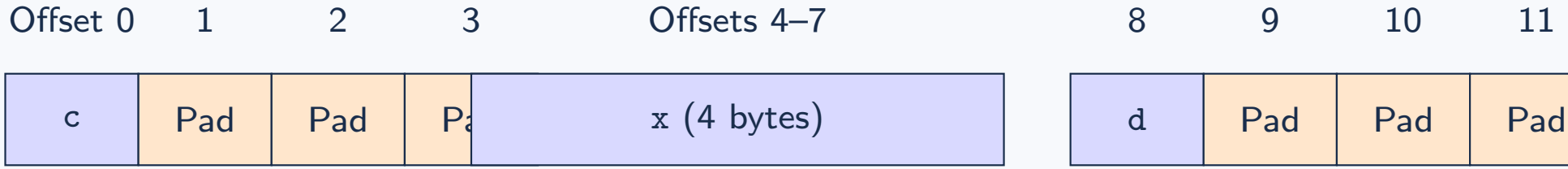
D. It must equal the sum of the member sizes, with no padding.

MCQ 5 — Why Does Padding Exist?

Correct Answer: C

The size depends on the implementation. C allows padding bytes to satisfy the alignment requirements of members.

One possible memory layout Assume `char` occupies 1 byte, `int` occupies 4 bytes, and `int` requires 4-byte alignment.

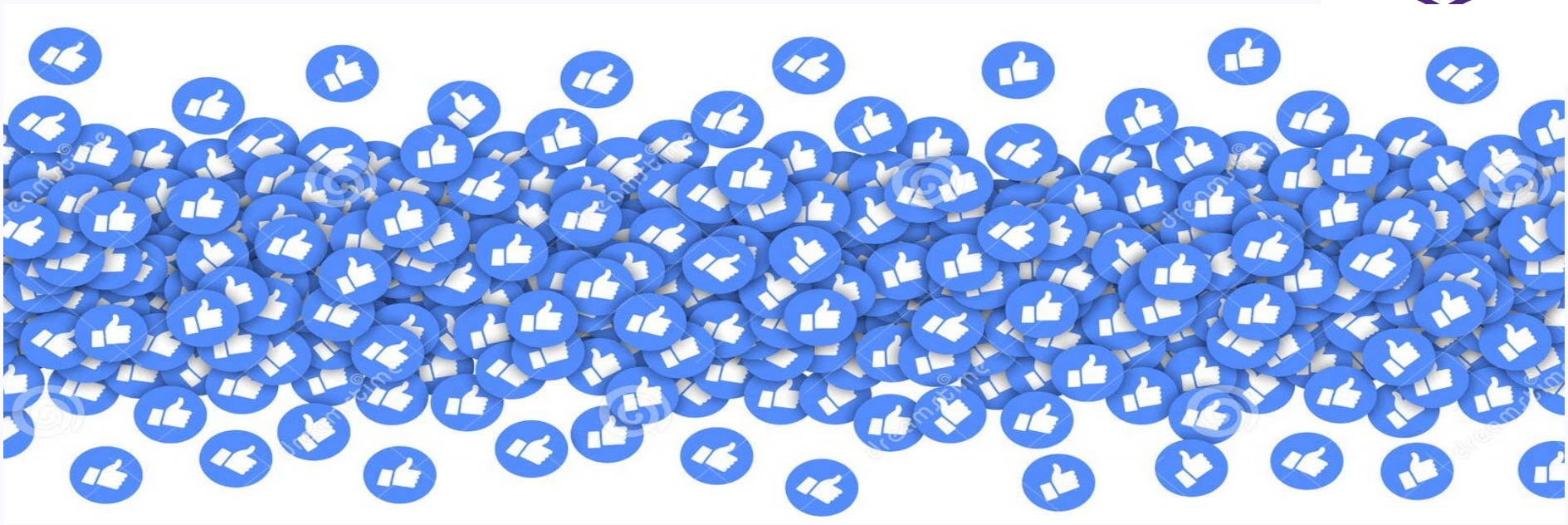


Count the Bytes

- c: 1 byte, Padding before x: 3 bytes, x: 4 bytes, d: 1 byte, Tail padding: 3 bytes

$$\boxed{1 + 3 + 4 + 1 + 3 = 12 \text{ bytes}}$$

Never assume that `sizeof(struct T)` is simply the sum of the sizes of its members.



Thank you for your kind attention!!